

Lab 3 Report

- The controller was designed using the subsumption architecture, where behaviours are organized into a hierarchy of competences. Higher-priority competences suppress lower-priority ones whenever they become active.
- The initial planned hierarchy was: Halt on black spot, Phototaxis, Obstacle avoidance, Random walk
- During testing, it became apparent that placing phototaxis above obstacle avoidance created undesirable behaviour. When obstacles were located close to the light source, the robot repeatedly attempted to move directly towards the light, even when an obstacle blocked the path.
- To solve this issue, the hierarchy was modified to:
 - Halt on black spot
 - Obstacle avoidance
 - Phototaxis
 - Random walk
- With this ordering, obstacle avoidance temporarily suppresses phototaxis whenever an obstacle is detected.
- Once the obstacle is no longer present, control is passed back to phototaxis, allowing the robot to continue moving towards the light. Resulting in smoother navigation and significantly reduces the chance of the robot becoming trapped near obstacles.
- Behaviour descriptions
- The step() function starts execution by calling the highest-priority competence (halt_on_black_spot()).
- If the halt condition is not satisfied, control is passed to the next competence (obstacle_avoidance()).
- If no obstacle is detected, control is passed to phototaxis().
- If no significant light source is detected, control is passed to random_walk().
- Each competence either:
 - Executes its own behaviour and suppresses lower layers, or
 - Passes control to the next lower-priority competence.
 - This creates a decentralized control structure where behaviours coordinate directly with one another rather than relying on a central decision-making module.
- The design is modular and easily extensible, since an additional competence can be inserted into the hierarchy by adding a new function and linking it into the existing chain of behaviours.
- This architecture was tested with several robots and they all seemed to reach the goal eventually
- An additional possible improvement to the current code is to add a limit to how long a robot stays stuck in a certain position, so if the robot is stuck between obstacles for n steps, it initiates random walk.